

# The Capstone Case Study

*Vendor Managed Inventory — a real, multi-process business problem that uses everything you've learned.*

# The business problem

- The Company agrees to perform a Vendor Managed Inventory (VMI) service for two retail customers: My Little Store (MLS) and The Other Guy (TOG).
- The company's auditor requires a copy of the original data — so a copy of every inbound file must move to an archive directory automatically.
- All processing must be automated, with no manual handling of data. Inbound files are placed in `/opt/student/fscoll`.
- There are many valid solutions to this case study — the one in the course guide builds bootstrapping on a File System Adapter into a business process, rather than scheduling a File System Adapter configuration directly.

# Three business processes make up this solution

*Build them in this order — later ones depend on earlier ones being checked in*

| # | Business process                 | What it does                                                                                                      |
|---|----------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1 | LittleStore.BP / OtherGuy.BP     | Determines whether each customer needs to order more items (built first — Main.BP invokes these)                  |
| 2 | Main.BP                          | Archives an audit copy, determines the customer ID, routes to the correct per-customer BP, writes the result file |
| 3 | KICKOFF.BP (File System Adapter) | Collects files directly from the inbound folder and bootstraps Main.BP to start the whole flow                    |

# Main.BP: archive, then route by customer

*Stencils: Start/End, Choice Start/End, 3× Sequence Start/End, XML Encoder, 2× Invoke BP, 2× File System Adapter*

| Step                                      | Configuration                                                                                                                                |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| File System Adapter (archive step, FIRST) | Action = Extraction; Bootstrap = No; ExtractionFolder = /opt/student/fsarchive — writes the auditor's required copy before any routing runs; |
| Rule — Check for MLS                      | XPath: //INVENTORY/CUST_ID/text()='My Little Store'                                                                                          |
| Rule — Check for TOG                      | XPath: //INVENTORY/CUST_ID/text()='The Other Guy'                                                                                            |
| Invoke Business Process (MLS path)        | Invoke_mode = Sync; WFD_Name = LittleStore.BP                                                                                                |
| Invoke Business Process (TOG path)        | Invoke_mode = Sync; WFD_Name = OtherGuy.BP                                                                                                   |
| File System Adapter (final step)          | Action = Extraction; Bootstrap = No; ExtractionFolder = /opt/student/fsoutput                                                                |

# Per-customer business processes: order-more logic

*Build LittleStore.BP first, then Save As to create OtherGuy.BP with the threshold changed*

| Customer BP    | Rule (XPath)                    | Assign on true / not true                                                                              |
|----------------|---------------------------------|--------------------------------------------------------------------------------------------------------|
| LittleStore.BP | sum(INVENTORY/PRODUCT/QTY) > 10 | not true → Constant "Order More Items" → Results   true → Constant "Do Not Order More Items" → Results |
| OtherGuy.BP    | sum(INVENTORY/PRODUCT/QTY) > 20 | Same pattern as LittleStore.BP, with the threshold changed to 20                                       |

# Kicking off the flow: File System Adapter bootstrap only

*One File System Adapter does the whole job*

1

## KICKOFF.BP (File System Adapter)

Action = Collection; Bootstrap = Yes; CollectionFolder = /opt/student/fscoll; DeleteAfterCollect = Yes; InitialWorkFlowid = main.bp — collects the inbound file directly and starts Main.BP running.

2

## Main.BP takes it from there

Main.BP's own first step (previous slide) immediately writes the audit copy to fsarchive, then routes by customer as before — no separate archive folder or hand-off process needed.

*Tip: run KICKOFF.BP manually via Business Process > Manager > Execution Manager for testing — no input document needed, since Collection mode scans its own folder. In production, schedule it instead.*

# Live Capstone Build

# Field-mapping configuration: Assign Service with XPath

*Type each expression live — this is field mapping, done natively*

| What you're configuring | Exact expression to type live                                                                           | What it demonstrates                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Routing rule (Choice)   | <code>//INVENTORY/CUST_ID/text()='My Little Store'</code>                                               | Reading a field out of incoming XML to drive a decision                              |
| Routing rule (Choice)   | <code>//INVENTORY/CUST_ID/text()='The Other Guy'</code>                                                 | Same pattern, second customer                                                        |
| Assign — computed field | <code>OrderTotal = sum(INVENTORY/PRODUCT/PRICE)</code>                                                  | Computed field from multiple source values — same idea as Session 13's ExtendedPrice |
| Assign — computed field | <code>OutputFile =<br/>concat('Proc#',//WORKFLOW_ID/text(),'_Total_',//OrderTotal/text(),'.txt')</code> | Building an output field from several mapped inputs at once                          |



# Sub-process modes and closing the loop

*Same scenario, picking up after the output file is confirmed*

1

Show Invoke Sub-Process in sync mode (used here) vs. async, and explain the best-practice choice given this process needs a single-threaded, ordered result.

2

Close with the Release Service to clean up process data no longer needed.

# Hands-On Exercise

*Build the Vendor Managed Inventory capstone*

In your own lab environment, build all three business processes: LittleStore.BP, OtherGuy.BP, Main.BP (with the archive step as its first action), and KICKOFF.BP.

- Use the exact XPath rules and thresholds from lecture (MLS:  $\text{sum}(\text{QTY}) > 10$ ; TOG:  $\text{sum}(\text{QTY}) > 20$ ).
- Verify the setup directories exist (/opt/student/fsarchive and /opt/student/fsoutput — fscoll is your existing inbound drop folder).
- Copy the MLS and TOG sample data files into /opt/student/fscoll, run KICKOFF.BP manually, and confirm the whole chain fires end to end.

# Best Practices Recap

*Three services you've used throughout this course — used correctly.*

# Best practices: Assign Service

SOURCED

- Use the Assign Service (not repeated Assign activities) when multiple assignments need to be made consecutively.
- Reducing the number of steps in a process can increase performance — a documented test with two Assign activities versus the same values in one Assign Service ran in approximately half the time.
- The Assign Service is found in the Sync Mode Stencil.

# Best practices: Invoke Sub-Process Service

SOURCED

*Invoke Sub-Process and Invoke Business Process are different names for the same service*

| Mode                   | Behavior / when to use                                                                                                                                                                                                |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synchronous            | Parent suspends until the child returns data, and is notified if the child errors. Use for single-threaded, ordered flows — e.g., today's capstone, or large files like Catalogs (832) / Product Activity Data (852). |
| Asynchronous (default) | Parent and child process independently and simultaneously; the parent is NOT notified of child errors. Typical for most applications — allows concurrent, multi-threaded processing.                                  |
| Inline                 | Subprocess runs as part of the parent, sharing process data. Good for high volumes, since it minimizes the number of threads created.                                                                                 |
| Embedded               | Subprocess runs without persistence — no record in Application tracking.                                                                                                                                              |

## Performance tip: `message_to_child` / `message_to_parent`

- By default, ALL process data passes from the parent process to its subprocess when using Invoke Sub-Process — that can be expensive on large payloads.
- In sync mode, create a special tag called `message_to_child` in the parent, and append only the data the subprocess actually needs under it — Invoke Sub-Process will pass only this tag.
- `message_to_parent` works the same way in reverse — only data under this tag is passed back to the parent, limiting the return payload.
- Using these tags can provide a significant performance improvement — directly reinforcing Session 12's performance-tuning discipline.

# Best practices: Release Service

- The Release Service is a system service that removes extraneous data from process data — useful for cleaning up large payloads to aid performance.
- It ships with no predefined parameters — you must use the Advanced Service Editor to create a parameter named TARGET.
- TARGET's value is an XPath expression selecting the process-data nodes to release — even though it's an XPath expression, you do NOT need to check the "Use XPATH" box.
- Multiple process-data tags can be released in one Release statement, reducing the overall number of steps in the business process.

# Hands-On Exercise 2

*Apply best practices to your capstone build*

In your Main.BP, replace any repeated Assign activities with a single Assign Service and confirm the process still produces the correct output.

- Confirm your Invoke Business Process steps use Sync mode (as required for this ordered, single-threaded flow) and be ready to explain why Async would be wrong here.
- Add a Release Service to the end of your capstone process; create a TARGET parameter via the Advanced Service Editor releasing a process-data node you no longer need.
- Re-run the full flow end to end and confirm the output file is unchanged.